PyQGIS - plugins de interface

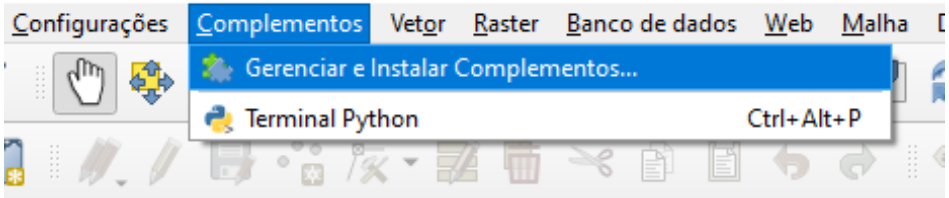
Disciplina: Programação aplicada à engenharia cartográfica

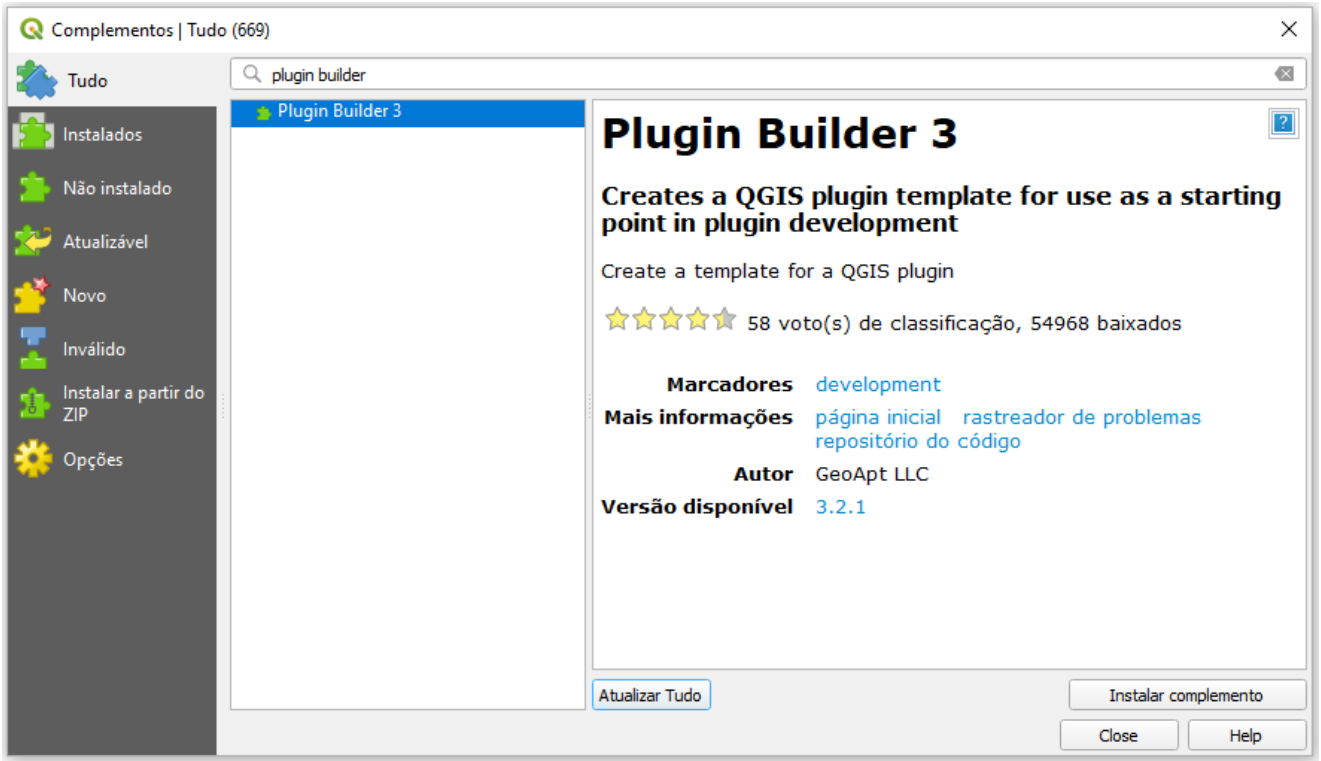
Maurício C. M. de Paulo - D.Sc.

19 de agosto de 2026

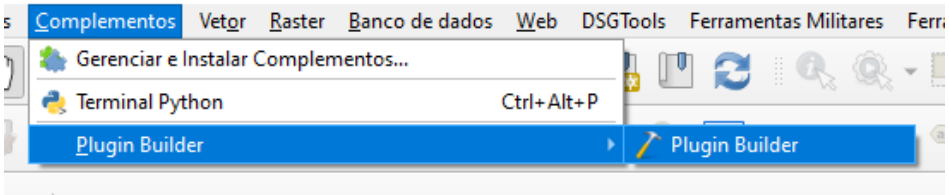
Sumário

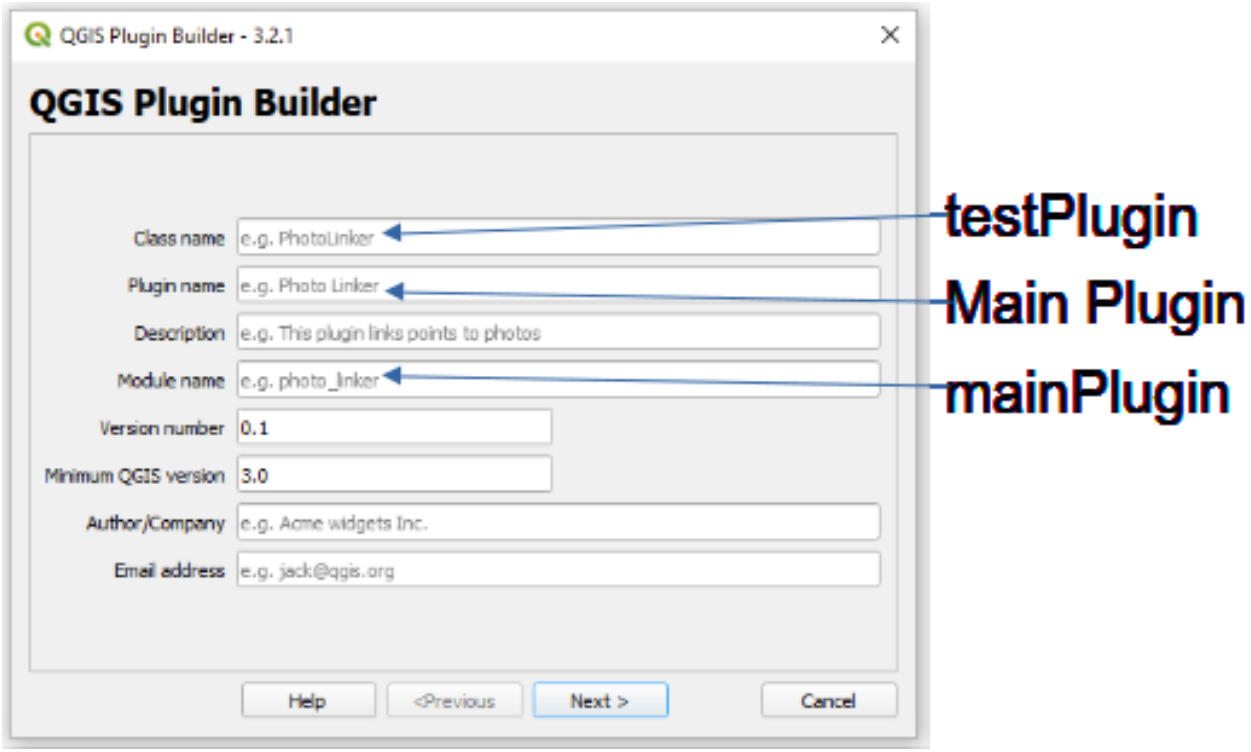
Plugin Builder: Passo a passo



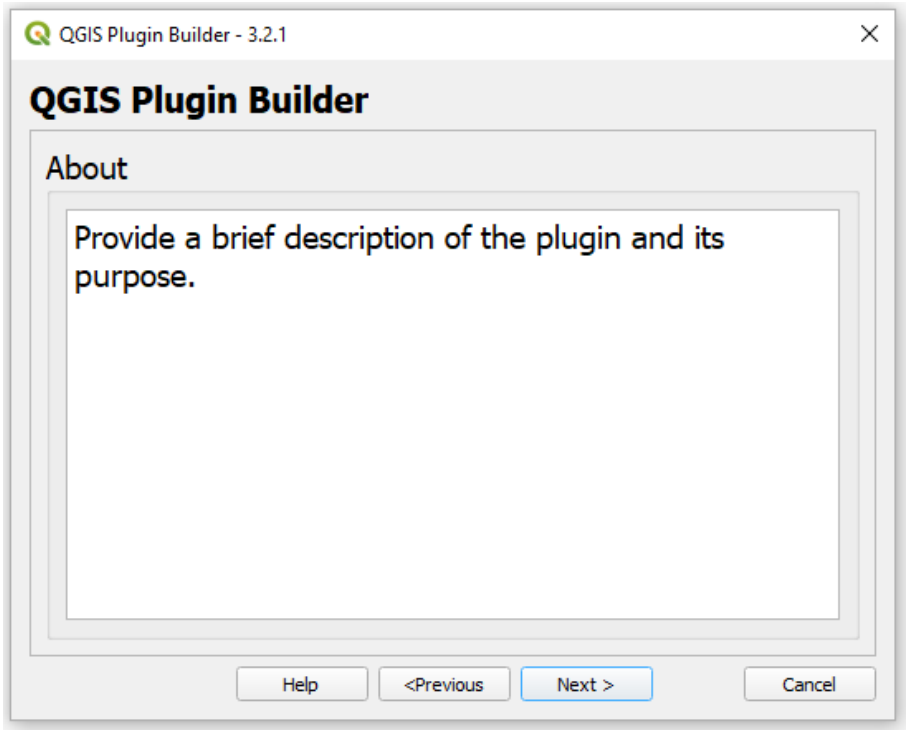


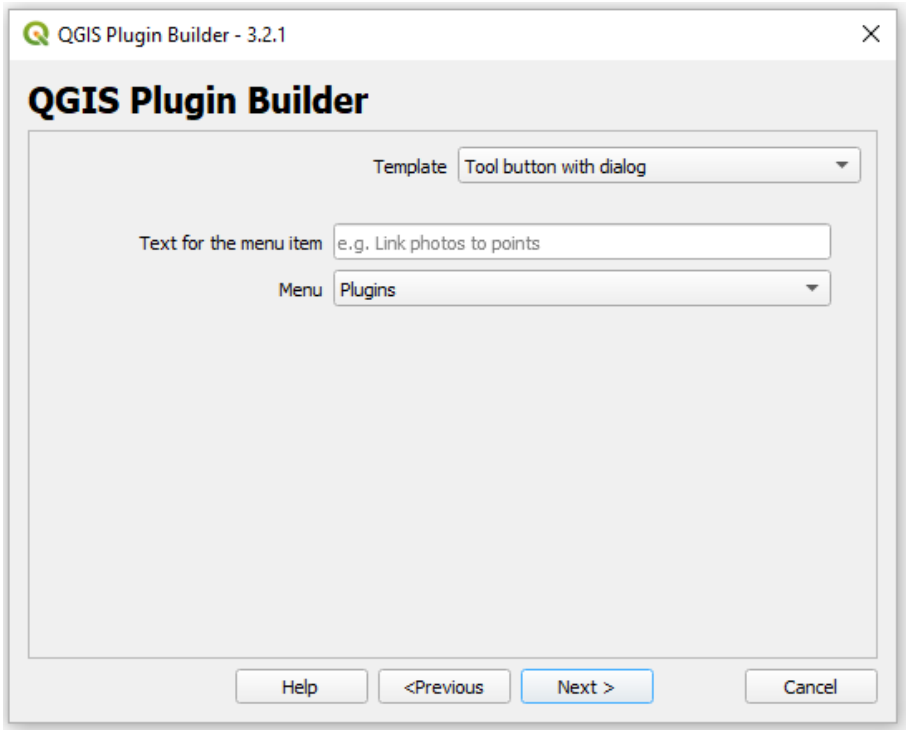
Plugin Builder: Passo a passo



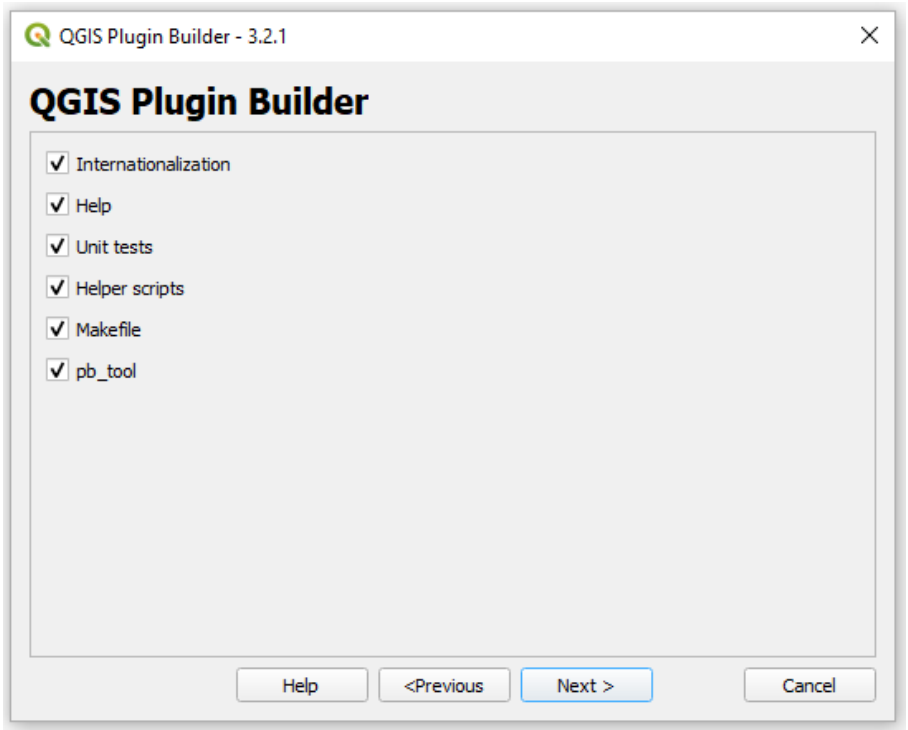


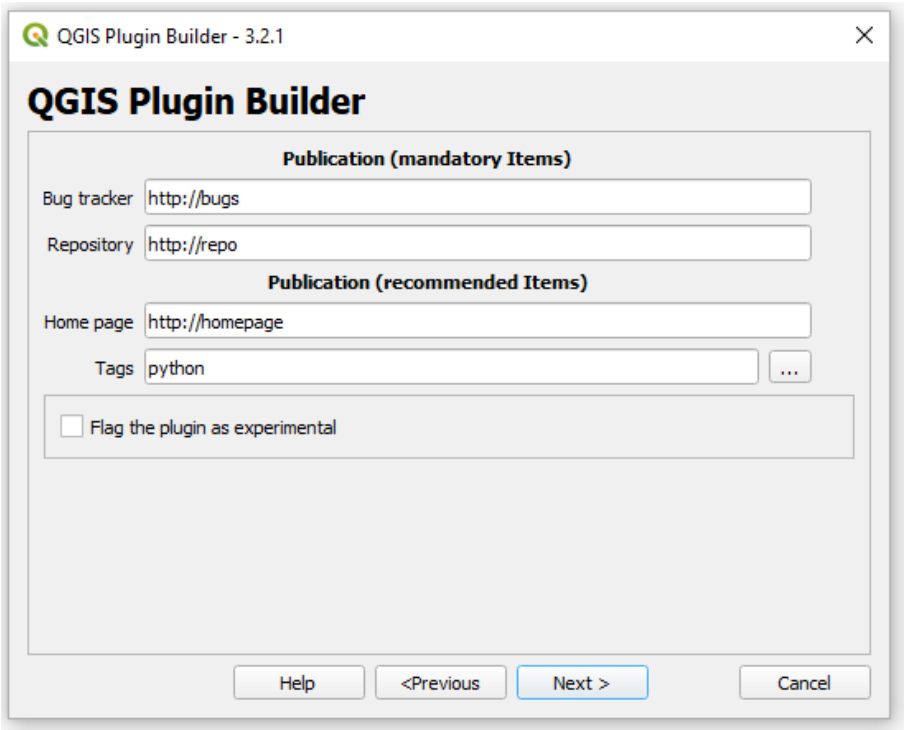
Plugin Builder: Passo a passo





Plugin Builder: Passo a passo





Plugin Builder: Onde salvar?

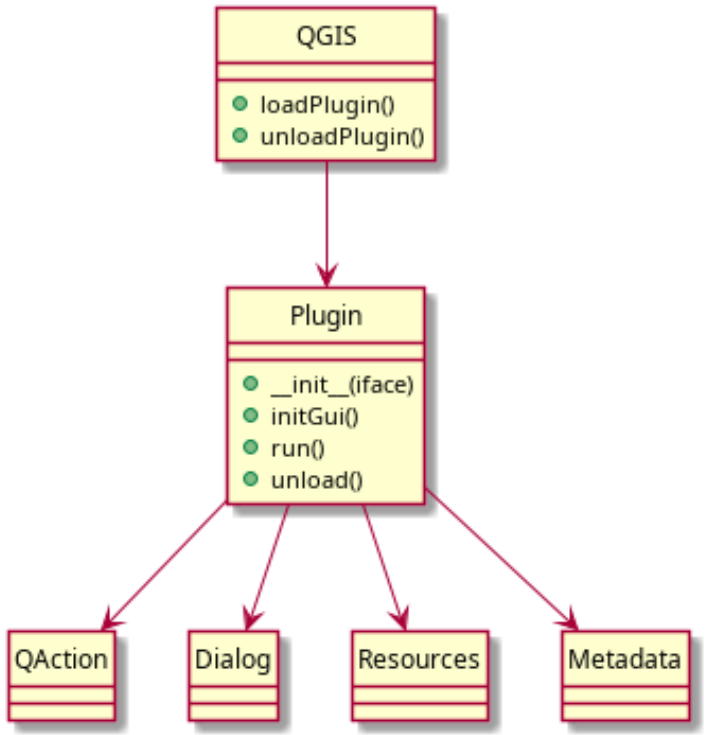
- Onde salvar?
- LINUX
- `/home/username/.local/share/QGIS/QGIS3/profiles/default/python/plugins/`
- WINDOWS
- `C:\Users\<user>\AppData\Roaming\QGIS\QGIS3\profiles\default\python\plugins\`

```
from qgis.core import QgsApplication
print(QgsApplication.qgisSettingsDirPath())
```

Estrutura básica de um plugin QGIS

Principais componentes de um plugin PyQGIS:

- Classe principal do plugin
- Interface do QGIS (`iface`)
- Arquivo de metadados
- Interface gráfica (Qt / .ui)



Criação de Plugin: Estrutura básica do plugin

- __init__.py = The starting point of the plugin. It has to have the classFactory() method and may have any other initialisation code.
- mainPlugin.py = The main working code of the plugin. Contains all the information about the actions of the plugin and the main code.
- resources.qrc = The .xml document created by Qt Designer. Contains relative paths to resources of the forms.
- resources.py = The translation of the .qrc file described above to Python.
- form.ui = The GUI created by Qt Designer.
- form.py = The translation of the form.ui described above to Python.
- metadata.txt = Contains general info, version, name and some other metadata used by plugins website and plugin infrastructure.

`https://docs.qgis.org/3.34/en/docs/pyqgis_developer_cookbook/plugins/plugins.html`

Criação de Plugin: GitHub

Objetivo: Colocar a pasta .git e o código gerado pelo Plugin Builder na pasta do QGIS.

GitHub Desktop + QGIS + Plugin Builder

- Criar pasta do plugin no Plugin Builder
- Criar repositório GIT no GitHub
- Baixar (Clone) o repositório para a máquina local
- Copiar os arquivos de dentro da pasta do plugin para o repositório
- Mover pasta do repositório para a pasta de plugins do QGIS
- No GITHUB Desktop, adicionar repositório local e apontar para a pasta do plugin dentro da estrutura do QGIS

Sumário

Depuração de plugins QGIS — Visão geral

Depurar plugins QGIS pode ser desafiador porque o código roda **dentro do QGIS**.

Estratégia:

- Usar o **VSCodium** como debugger
- Conectar ao processo do QGIS
- Inserir breakpoints no código do plugin

Ferramenta principal:

- `debugpy` (debugger Python)

Passo 1 — Inserir debugpy no plugin

Adicione no código do plugin (ex: no `initGui()` ou início do run):

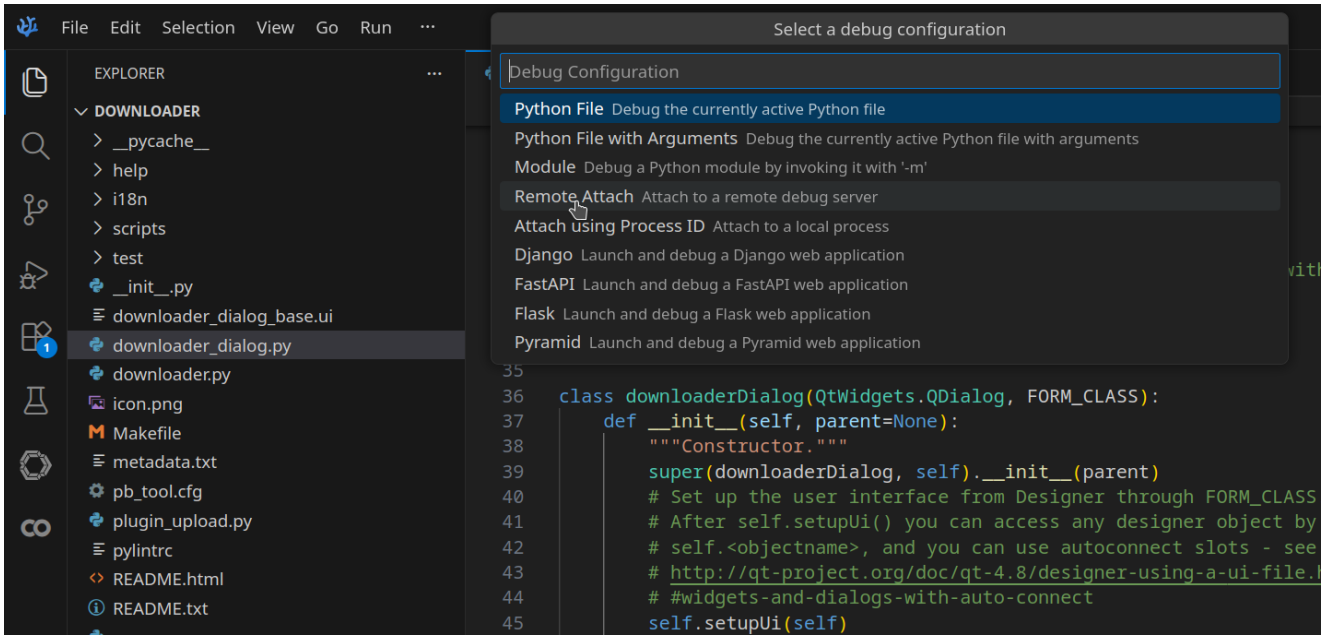
```
import debugpy

debugpy.listen(("localhost", 5678))
print("Aguardando debugger...")

debugpy.wait_for_client()
print("Debugger conectado!")
```

Efeito:

- QGIS pausa execução
- Aguarda conexão do VSCodium

Passo 2 — Configurar VSCodium

Ao iniciar a depuração, escolha "Remote attach".



Depois:

- Abrir o plugin no QGIS
- Executar a depuração.

Passo 3 — Depuração na prática

Fluxo de uso:

1. Abrir QGIS
2. Executar o plugin
3. QGIS pausa em `wait_for_client()`
4. Iniciar debug no VSCodium
5. Continuar execução

Agora você pode:

- Colocar breakpoints
- Inspecionar variáveis
- Executar passo a passo

Dica:

Remova os comandos antes de publicar o plugin.